TP 2 : Base de données

Soit le schéma d'une base de données d'un magasin :

Schéma relationnel

produit(*id_produit*, *label*, *prix_unitaire*, *stock*)
client(*id_client*, *nom*, *contact*)
commande(*id_commande*, *#id_client*, *date_commande*, *total_commande*)
ligne_commande(*#id_commande*, *#id_produit*, *quantity*)

id_client	*nom*	*contact*
1	Ali	0606612358
2	Ahmad	0617815611
3	Fatima	0628101123
4	Omar	0639123581
5	Safa	0641011235
6	Rim	0651123581
7	Ibrahim	0661235813
…	…	…

Table 1 – Extrait de la relation client

id_produit	*label*	*prix_unitaire*	*stock*
1	Stylo Bille	2.00	100
2	Cahier 100 pages	4.50	50
3	Classeur A4	7.00	30
4	Surligneur	2.50	75
6	Pochette plastique	0.60	200
7	Règle 30cm	3.50	80
…	…	…	…

Table 2 – Extrait de la relation produit

id_commande	*id_client*	*date_commande*	*total_commande*
1	1	2025-01-01	20.50
2	2	2025-01-01	152.00
3	1	2025-01-01	136.50
4	3	2025-01-01	88.20
5	5	2025-01-01	40.00
6	4	2025-01-02	288.00
7	1	2025-01-02	29.30
…	…	…	…

Table 3 – Extrait de la relation commande

id_commande	*id_produit*	*quantity*
1	1	4
1	2	2
1	7	1
2	3	2
2	8	3
2	10	1
3	9	3
…	…	…

Table 4 – Extrait de la relation ligne_commande

1. Créer les tables mentionnées ci-dessus et y insérer les données.
2. Sélectionner les labels des produits en rupture de stock.
3. Afficher l'historique (date et total) des commandes du client dont le contact est 0617815611.
4. Afficher les labels et les quantités des produits de chaque commande.
5. 5.1 Afficher la quantité en stock du produit « Papier imprimante A4 (500 feuilles) » ;
   5.2 Afficher la quantité vendue du même produit.
6. Calculer le total des commandes et la TVA associée.
7. Calculer le total à payer pour une commande de 5 « Calculatrice scientifique ».
8. 8.1 Afficher les identifiants des produits déjà commandés ;
   8.2 En déduire les labels et les quantités en stock des produits jamais commandés.
9. 9.1 Afficher le nom, le contact et le montant total des commandes de chaque client ;
   9.2 Afficher le nom et le contact du client qui a le total maximal des commandes.
10. 10.1 Afficher le total des commandes de chaque client ;
    10.2 En déduire la moyenne de ces totaux ;
    10.3 Lister les identifiants des clients dont le total des commandes dépasse cette moyenne.

Résolution

Question 1 : création des tables

```sql
CREATE TABLE IF NOT EXISTS client (
    id_client INTEGER PRIMARY KEY,
    nom       TEXT NOT NULL,
    contact   TEXT UNIQUE NOT NULL
);

CREATE TABLE IF NOT EXISTS produit (
    id_produit    INTEGER PRIMARY KEY,
    label         TEXT NOT NULL,
    prix_unitaire REAL NOT NULL CHECK (prix_unitaire >= 0),
    stock         INTEGER NOT NULL
);

CREATE TABLE IF NOT EXISTS commande (
    id_commande    INTEGER PRIMARY KEY,
    id_client      INTEGER NOT NULL,
    date_commande  TEXT NOT NULL,
    total_commande REAL NOT NULL,
    FOREIGN KEY (id_client) REFERENCES client(id_client)
        ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS ligne_commande (
    id_commande INTEGER,
    id_produit  INTEGER,
    quantity    INTEGER NOT NULL,
    PRIMARY KEY (id_commande, id_produit),
    FOREIGN KEY (id_commande) REFERENCES commande(id_commande)
        ON DELETE CASCADE,
    FOREIGN KEY (id_produit) REFERENCES produit(id_produit)
);
```

Question 1 : insertion des données

```sql
INSERT INTO client VALUES
(1, 'Ali', '0606612358'),
(2, 'Ahmad', '0617815611'),
(3, 'Fatima', '0628101123'),
(4, 'Omar', '0639123581'),
(5, 'Safa', '0641011235'),
(6, 'Rim', '0651123581'),
(7, 'Ibrahim', '0661235813');
-- ...

INSERT INTO produit VALUES
(1, 'Stylo Bille', 2.00, 100),
(2, 'Cahier 100 pages', 4.50, 50),
(3, 'Classeur A4', 7.00, 30),
(4, 'Surligneur', 2.50, 75),
(6, 'Pochette plastique', 0.60, 200),
(7, 'Règle 30cm', 3.50, 80);
-- ...

INSERT INTO commande VALUES
(1, 1, '2025-01-01', 20.50),
(2, 2, '2025-01-01', 152.00),
(3, 1, '2025-01-01', 136.50),
```

```sql
(4, 3, '2025-01-01', 88.20),
(5, 5, '2025-01-01', 40.00),
(6, 4, '2025-01-02', 288.00),
(7, 1, '2025-01-02', 29.30);
-- ...

INSERT INTO ligne_commande VALUES
(1, 1, 4),
(1, 2, 2),
(1, 7, 1),
(2, 3, 2),
(2, 8, 3),
(2, 10, 1),
(3, 9, 3);
-- ...
```

Question 2

```sql
SELECT label FROM produit WHERE stock = 0;
```

Question 3

```sql
SELECT date_commande, total_commande
FROM client
JOIN commande ON client.id_client = commande.id_client
WHERE contact = '0617815611';
```

Question 4

```sql
SELECT id_commande, label, quantity
FROM ligne_commande
JOIN produit ON produit.id_produit = ligne_commande.id_produit
ORDER BY id_commande;
```

Question 5.1

```sql
SELECT stock FROM produit
WHERE label = 'Papier imprimante A4 (500 feuilles)';
```

Question 5.2

```sql
SELECT SUM(quantity) AS qtt_vendue
FROM ligne_commande
JOIN produit ON produit.id_produit = ligne_commande.id_produit
WHERE label = 'Papier imprimante A4 (500 feuilles)';
```

Question 6

```sql
SELECT SUM(total_commande) AS total,
       0.2 * SUM(total_commande) AS tva     -- taux de TVA : 20 %
FROM commande;
```

Question 7

```sql
SELECT 5 * prix_unitaire AS prix_total
FROM produit
WHERE label = 'Calculatrice scientifique';
```

Question 8.1

```sql
SELECT DISTINCT id_produit FROM ligne_commande;
```

On obtient l'ensemble des identifiants des produits déjà commandés (`DISTINCT` supprime les doublons).

Question 8.2

```sql
SELECT label, stock
FROM produit
WHERE id_produit NOT IN (SELECT DISTINCT id_produit
                         FROM ligne_commande);
```

On réutilise la requête de la question 8.1 comme sous-requête : on garde les produits dont l'identifiant n'appartient pas à l'ensemble des produits déjà commandés.

Question 9.1

```sql
SELECT client.id_client, nom, contact,
       SUM(total_commande) AS totaux
FROM client
JOIN commande ON client.id_client = commande.id_client
GROUP BY client.id_client;
```

Question 9.2

```sql
SELECT client.id_client, nom, contact,
       SUM(total_commande) AS totaux
FROM client
JOIN commande ON client.id_client = commande.id_client
GROUP BY client.id_client
ORDER BY totaux DESC
LIMIT 1;
```

Question 10.1

```sql
SELECT id_client, SUM(total_commande) AS totaux
FROM commande
GROUP BY id_client;
```

Question 10.2

```sql
SELECT AVG(totaux) AS moyenne
FROM (SELECT id_client, SUM(total_commande) AS totaux
      FROM commande
      GROUP BY id_client);
```

La requête de la question 10.1 devient une sous-requête dans le `FROM` : elle fournit une table temporaire des totaux par client, dont on calcule la moyenne.

Question 10.3

```sql
SELECT id_client, SUM(total_commande) AS totaux
FROM commande
GROUP BY id_client
HAVING totaux > (SELECT AVG(totaux)
                 FROM (SELECT id_client, SUM(total_commande) AS totaux
                       FROM commande
                       GROUP BY id_client));
```

On reprend la requête de la question 10.1 et on filtre les groupes avec `HAVING` ; la valeur de comparaison est la moyenne calculée à la question 10.2.